Unqualified enumerators in case labels

Document number:	P0284R0
Date:	2016-02-14
Project:	Programming Language C++, Evolution Working Group

Reply-to:	James Touton <bekenn@gmail.com>

Table of Contents

Introduction

This paper proposes allowing unqualified name lookup to find (scoped or unscoped) enumerators when the type of the condition of the enclosing switch statement is an enumeration. Example:

```
enum class Foo
{
    Bar,
    Baz
};

namespace N
{
    enum Alehouse
    {
        Pub,
        Bar,
        Tavern
    }
}

int f(Foo foo, N::Alehouse h)
{
    switch (foo)
    {
    case Bar:       // ok: finds Foo::Bar
        return 1;
    }

    switch (h)
    {
    case Bar:       // ok: finds N::Alehouse::Bar
        return 2;
    }

    return -1;
}
```

Motivation and Scope

This proposal aims to reduce unnecessary verbosity. Switch statements and enumerations are very frequently used together. A case label's constant expression must be convertible to the (adjusted) type of the switch statement's condition; when the condition is an enumeration, each case label's constant expression must be convertible to that enumeration. When an enumerator belonging to that enumeration appears in a case label, it therefore seems unnecessarily pedantic to require the programmer to explicitly qualify the enumerator name when the enumerator is not presently visible in the enclosing scope.

```
enum class Color
{
    Black, Blue, Green, Cyan, Red, Magenta, Yellow, White
};
```

```
int color_code(Color color)
{
    // Under current rules, every enumerator here must
    // be prefixed with the enumeration name even though
    // no other values are possible without explicit
    // conversions.
    switch (color)
    {
    case Color::Black:
        return 0x000000;
    case Color::Blue:
        return 0x0000FF;
    case Color::Green:
        return 0x00FF00;
    case Color::Cyan:
        return 0x00FFFF;
    case Color::Red:
        return 0xFF0000;
    case Color::Magenta:
        return 0xFF00FF;
    case Color::Yellow:
        return 0xFFFF00;
    case Color::White:
        return 0xFFFFFF;
    }
}
```

Impact On the Standard

This proposal extends the visibility of enumerators to cover case labels when the condition of the associated switch statement is of the type of the enumeration.

Some pathological examples of existing code could change in meaning. In order for this to happen, an existing case label would need to reference a named constant in an enclosing namespace (outside of the function definition) that has the same name as an enumerator and a value distinct from that enumerator. This should be extremely rare, and may actually be indicative of a bug in existing code.

Design Decisions

Name hiding

This proposal extends the potential scope of enumerators to cover case labels in switch statements where the codition of the switch statement has the same type as the enumeration. This carries with it new potential for name collisions when the name of an enumerator already carries different meaning in the scope of the case label. This paper takes the position that an enumerator name appearing in a case label is more likely to refer to the enumerator than to another entity, but gives deference to function-local entities that can't be referenced any other way. An enumerator name is therefore allowed to hide entities at class and namespace scope, but is itself hidden by entities at function and block scope.

Example:

```
enum class Color
{
    Black, Blue, Green, Cyan, Red, Magenta, Yellow, White
};

constexpr int Yellow = 42;

int color_code(Color color)
{
    int Black = 27;

    switch (color)
    {
    case Black:             // error: function-local Black, not Color::Black
        return 0x000000;
    case Yellow:            // ok: Color::Yellow
        return 0xFFFF00;
    // ...
    }

    return -1;
}
```

Technical Specifications

- When a case label appears in a switch statement and the adjusted type of the switch statement's condition is an enumeration, the enumerators belonging to that enumeration are visible in case label's *constant-expression*.
- Enumerators made visible in a case label hide competing names at class scope and namespace scope.

Wording

All modifications are presented relative to N4567.

Add the following paragraph to §3.3.1 [basic.scope.declarative]:

> The potential scope of an enumerator additionally includes some case labels as described in 6.4.2.

Insert the following paragraph after §3.3.10 [basic.scope.hiding] paragraph 4:

> Within the *constant-expression* of a case label where the potential scope of an enumerator has been extended to include the *constant-expression* (6.4.2), a name declared at class or namespace scope can be hidden by the enumerator, and the enumerator can be hidden by a name declared at function or block scope.

Insert the following paragraph after §6.4.2 [stmt.switch] paragraph 2:

> When the condition is of enumeration type, the potential scope of the enumerators of the enumeration is extended to include the *constant-expression*s of the `switch` statement's associated `case` labels. [*Example:*
>
> ```
> enum class color { red, yellow, green, blue };
> bool reddish(color col)
> {
>     switch (col)
>     {
>     case red:          // ok: color::red
>     case yellow:       // ok: color::blue
>         return true;
>     default:
>         return false;
>     }
> }
> ```
>
> *—end example*]